

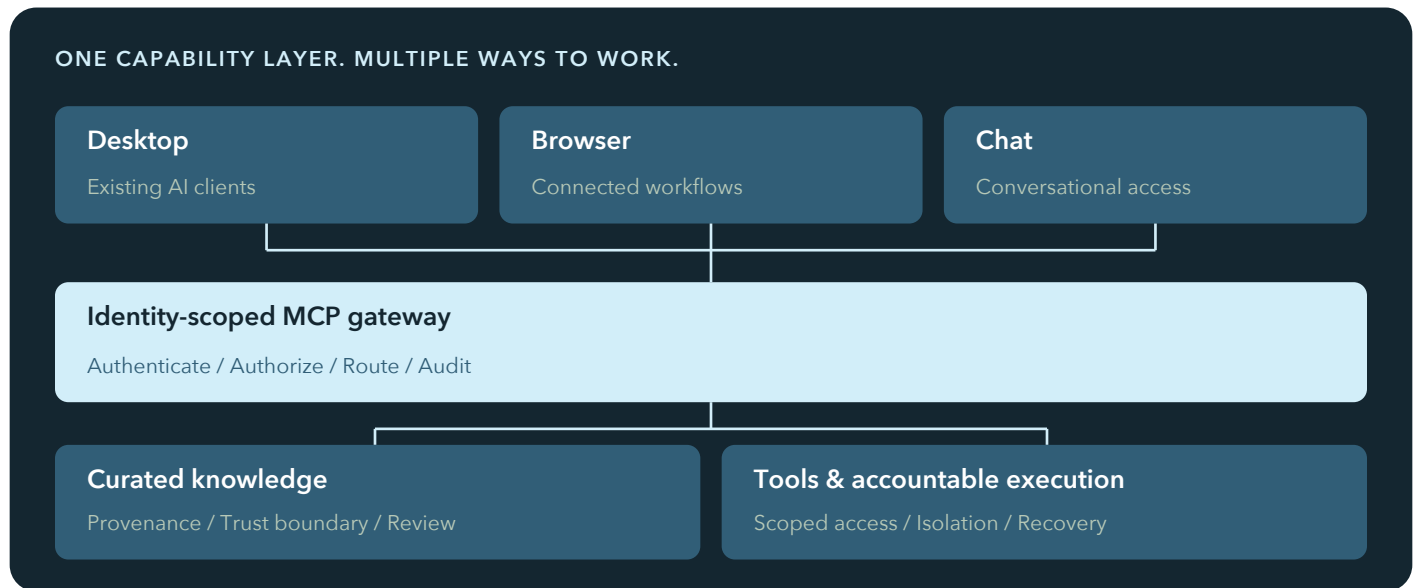
Elygent

AI PLATFORM DEVELOPMENT PILOT

Miami, FL | osmarmorales1@gmail.com | LinkedIn

Frontier AI. Shared team capability.

A connected experience across AI clients, tools, and curated knowledge. My goal is to absorb the complexity of a changing tool landscape and make useful capabilities available to the whole team.



Conceptual system view. Existing runtimes execute work; shared capabilities preserve identity and authorization across approved surfaces.

01 THE PRODUCT STRATEGY

Learn once. Make it useful to everyone. Explore emerging tools, evaluate where they improve a real workflow, package reusable capabilities, enable colleagues, and use operational feedback to improve the system. The platform should make people more capable without requiring everyone to become an integration expert.

One connected capability layer. Existing harnesses keep their strengths. Elygent provides shared access to identity-scoped tools, curated knowledge, and accountable work across desktop, browser, and chat surfaces.

Current scope. Owner-focused development pilot. The useful evidence is implemented integration and governance patterns, backed by a separate runnable portfolio reference.

The decisions behind the interface.

OSMAR MORALES / AI PLATFORM ENGINEERING

01 Identity before access

Resolve each client to an authorized person or business scope. The Node.js MCP gateway implements token scope, audience, and expiry checks, route restrictions, and negative tests for unauthorized access.

02 Knowledge with provenance

Store bounded facts and reusable decisions with source and trust boundaries. Checkpoint envelopes, duplicate handling, review dispositions, and expiry make durable knowledge distinct from raw conversation history.

03 Evidence before confidence

Check real model-to-tool execution in addition to service health. Stable failure fingerprints and deduplicated repair intake support recovery and review. Keep task ownership, approvals, and recovery responsibilities explicit.

02 IMPLEMENTATION & REFERENCE

Platform implementation: Node.js and the MCP SDK; Python integration workers; Linux, Docker Compose, systemd, Caddy, and PostgreSQL-backed services. Upstream runtimes retain their own execution responsibilities.

Runnable reference: A small Python module illustrates scoped reads, exact-action write approvals, replay protection, knowledge provenance, and expiry. It uses synthetic data, requires no provider account, and is separate from the Elygent runtime and employer systems.

18

Reference tests passed

Includes one-write-only behavior under concurrent approval replay.
Verification date: September 9, 2026.

03 TRADEOFFS & NEXT VALIDATION

A shared policy layer creates consistency while concentrating responsibility. Integrating multiple runtimes preserves flexibility while adding version-compatibility work. The reference uses inspectable in-memory state; distributed operation requires durable approvals and audit handling.

Next validation: reproducible workflow acceptance, identity-isolation and recovery tests, workload-specific latency and cost measurements, and clear operational ownership before broader deployment.